

Classes in Java

CS 101 Introduction to Programming II - CPU Spring 2026

Table of Contents

- [1. Objectives](#)
- [2. Hard pivot from C++ to Java](#)
- [3. Procedural vs. object-oriented programming](#)
- [4. Example: a light switch as message exchange between objects](#)
- [5. For comparison: C++ version of the Light Demo](#)
- [6. Example: procedural vs object-oriented Adder](#)
- [7. Everyday example of encapsulation: the car](#)
- [8. Classes and objects](#)
- [9. Glossary - check your understanding of basic terminology and examples!](#)
- [10. PRACTICE Create a Rectangle class](#)
- [11. Separate interface and implementation in Java](#)
- [12. Constructors](#)
- [13. Example: Counter constructors](#)
- [14. PRACTICE Constructor for the Rectangle class](#)
- [15. Solution: Constructor for the Rectangle class](#)
- [16. Attributes: public / class / static / void](#)

1. Objectives

- [] Understand procedural vs. object-oriented programming
- [] Define classes and create objects
- [] Use private and public access control
- [] Write member methods and constructors
- [] Understand encapsulation
- [] Distinguish interface from implementation in Java
- [] Build and test simple Java classes

We're learning just enough OOP to be able to build data structures.

2. Hard pivot from C++ to Java

- We are not switching because C++ is “bad” or Java is “better”.
- We are switching because: We have already learned:
 - arrays
 - memory
 - pointers
- That gave us **mechanical understanding**
- Now we want to focus on:
 - abstraction
 - data structures
 - behavior
- Our upcoming structures are not defined by memory layout but by allowed operations:
 1. list - add/remove/get/set
 2. stack - push/pop

3. queue - enqueue/dequeue
4. map - get/put/remove

- Now, after C++, you know what is abstracted away:

In C++, you controlled the data (through memory) directly. In Java, you control the behavior of the data (through methods).

- We move from **focusing on how data lives in memory** to **focusing on what data does through its interface**.

C++ (so far)	Java (now)
Where is the data?	What can I do with the data?
How is memory laid out?	What operations are allowed?
Can I access this address?	What methods define behavior?

- To be sure, all of this is also possible in C++ but if you don't know what you're doing...

...you can easily break the program by mismanaging memory, violating invariants, or exposing internal representation. In Java, low-lying details are largely hidden - all interaction is through well-defined interfaces, which define behavior only (via contract).

3. Procedural vs. object-oriented programming

- Most disciplines that grow over time are not just cobbled together: People identify patterns that are successful, explore and pursue them, and finally turn them into standards (until they fail).
- These patterns are also called "paradigms" (παράδειγματα). Two important programming paradigms are procedural and object-oriented programming.

- What does **procedural** mean? What does it achieve?

Procedural programming is centered on creating procedures or functions. Its main purpose is modularization.

- What is **object orientation**?

Object-orientation means that **everything is an object**. Each object has its own **state** and **behavior**.

- What does object orientation **achieve**?

Computation happens by message exchange between objects, like a conversation between agents. It enables **encapsulation** (of data), **inheritance** (of objects), and **polymorphism** (at compile-time or at run-time - Java chooses appropriate method based on object, e.g. you have an `Animal` class type: when you call `sound()`, the `Cat` or `Dog` version runs automatically depending on your object, not on the type).

- How does that look like?

In OOP, computation happens by sending messages to objects:

- `lamp.turnOn()` sends a message to a `lamp` object to turn on.

- `account.deposit(100)` sends a message to an account object to deposit \$100.

4. Example: a light switch as message exchange between objects

- Open OneCompiler.com/java and code along with me.
- Example 1: A light switch

```
// Light represents a device that knows how to perform actions
// It exposes behavior through public methods (its interface)
class Light {
    public void turnOn() {
        System.out.println("Light is on.");
    }

    public void turnOff() {
        System.out.println("Light is off.");
    }
}

// Switch does not control the Light directly.
// It does NOT change its internal state.
// Instead, it sends messages to the Light
class Switch {
    private Light light; // Reference to the Light object

    public Switch(Light l) {
        // The switch is connected to a specific Light
        // It knows where to send messages
        light = l;
    }

    public void flip(boolean on) {
        // The switch decides which message to send
        if (on) light.turnOn(); // send "turnOn" message to Light
        else light.turnOff(); // send "turnOff" message to Light
    }
}

public class LightDemo {
    public static void main(String[] args) {
        // Create a Light object (receiver of messages)
        Light lamp = new Light();
        // Create a Switch object and connect it to the Light
        Switch s = new Switch(lamp);

        // These calls don't manipulate Light directly
        // instead they send messages to the Switch
        s.flip(true); // Switch forwards "turnOn" to Light
        s.flip(false); // Switch forwards "turnOff" to Light
    }
}
```

- **Home assignment:** What would the procedural version of this program look like? Define a shared state and functions to turn the light on/off.

5. For comparison: C++ version of the Light Demo

- Here is a working program in C++ that does the same thing, demonstrating how Java abstracts the memory manipulation away by replacing pointers (open in browser: tinyurl.com/cpp-light-demo)

```

// Light represents a device that knows how to perform actions
// It exposes behavior through public methods (its interface)

#include <iostream>
using namespace std;

class Light {
public:
    void turnOn() {cout << "Light is on.\n";}
    void turnOff() {cout << "Light is off.\n";}
};

class Switch {
    Light *light; // knows where to send the message
public:
    Switch(Light *l) : light(l) {}; // flips the switch
    void flip(bool on) {
        if (on)
            light->turnOn(); // sends 'turnOn' message
        else
            light->turnOff(); // sends 'turnOff' message
    }
};

int main()
{
    Light lamp; // create a lamp
    Switch s(&lamp); // create a switch for the lamp
    s.flip(true); // throw the switch
    s.flip(false); // throw the switch
    return 0;
}

```

- Differences C++ vs. Java

Aspect	C++	Java
References / pointers	explicit pointers (Light *, &lamp, ->)	Implicit references (no &, no ->, use .)
Memory model	Stack objects + manual pointer use	Objects via references; automatic memory management
Method call syntax	light->turnOn()	light.turnOn()
Constructor style	Initializer list (: light(l))	Assignment in body (light = l;)
I/O	cout << ...	System.out.println(...)
Includes / imports	#include <iostream>	No import needed for basic output
Main function	int main()	public static void main(String[] args)
Object creation	Light lamp;	Light lamp = new Light();
Passing object to Switch	Switch s(&lamp);	Switch s = new Switch(lamp);
Access to members	public: / private: sections	public / private per member

Aspect	C++	Java
Abstraction level	Can operate at pointer or object level	Object-level only (no pointer access)
Error model	Possible pointer errors (dangling, null, etc.)	No pointer arithmetic; safer access model
Pedagogical emphasis	Memory + message passing via addresses	Encapsulation + behavior via methods

6. Example: procedural vs object-oriented Adder

- Now we're going into the opposite direction.
- This program represents an adder for different data types - it also documents the concept of **overloading**: both function names (add) and operator (+) are overloaded.

```
public class AddFunctions {
    public static int add(int a, int b) {
        return a + b;
    }

    public static double add(double a, double b) {
        return a + b;
    }

    public static String add(String a, String b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(5, 7));
        System.out.println(add(5.5, 6.5));
        System.out.println(add("five", "seven"));
    }
}
```

- Notice that the String class is already available without having to import anything (Java does not have a pre-processor).

PRACTICE From procedural to OOP

- Original program: tinyurl.com/adder-java.
- **Goal:** Move from static functions (procedural) to an object with behavior (OOP).
- **Instructions:**
 1. Create a class Adder
 2. Remove all static keywords (except in main)
 3. Turn each add function into an instance method
 4. In main, create a new Adder() object
 5. Call the methods through the object.
- Sample output:

```
12
12.0
fiveseven
```

- Upload your solution URL to Canvas (In-practice files).

Solution

```
class Adder {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public String add(String a, String b) {
        return a + b;
    }
}

public class AdderDemo {
    public static void main(String[] args) {
        Adder A = new Adder();

        System.out.println(A.add(5, 7));
        System.out.println(A.add(5.5, 6.5));
        System.out.println(A.add("five", "seven"));
    }
}
```

```
12.0
12.0
fiveseven
```

- Note:
 - In procedural style, all functions live in one scope.
 - In OOP style, related behavior is grouped in a class.
 - Java supports method overloading by parameter types.
- What happens if the first add method is declared private instead of public? What happens if all add methods are declared private?
 - If add(int, int) is private, it cannot be used from outside the class. Java then picks another accessible overloaded method (e.g., add(double, double)) using type conversion, so the program still runs but calls a different method.
 - If all add methods are private, none are accessible, so the program will not compile.
 - A private function cannot be called from another class. It is ignored in overload resolution outside the class.

7. Everyday example of encapsulation: the car

- What's part of the car's "user interface"? (What can you see/use?)

To use a car (as driver)

- Ignition switch
- Steering wheel
- Gas pedal
- Brake pedal
- Gear shift
- Dashboard display

- What's part of the car's "internal implementation"? (What's hidden?)

The mechanisms hidden from the user to be able to use it.

- Steering column
- Fuel injection
- Transmission system
- Brake hydraulics
- Most things outside the driver's field of vision

- Programming analogy:

- **public methods** = what the user can do
- **private fields** = internal state hidden from the user

- Encapsulation means:

hide implementation details and preserve a stable public interface.

8. Classes and objects

- What is a class? A class is three things: **keyword**, **code**, and **concept**.

- As a **keyword**:

As a **keyword** a class is a specifier for a user-defined **type** that you can use to cast data in any (digital) form you like. You can make your own categories.

So for example, the `Light` class allowed us to define a `lamp`. As an object, a `lamp` is like an integer or a string: memory + action.

- As **code**:

As code, a class specifies (aka encapsulates) the attributes (properties, data) and behavior (functions, methods) that an **object** of that class may have.

Example: The `Counter` class had one attribute, the `count`, one accessor/getter method, `getCount()`, and three update/setter methods, `increment` (with/without parameter), and `reset()`.

- As a **concept**:

As a concept, a class it is the **description** of an object, like a blueprint:

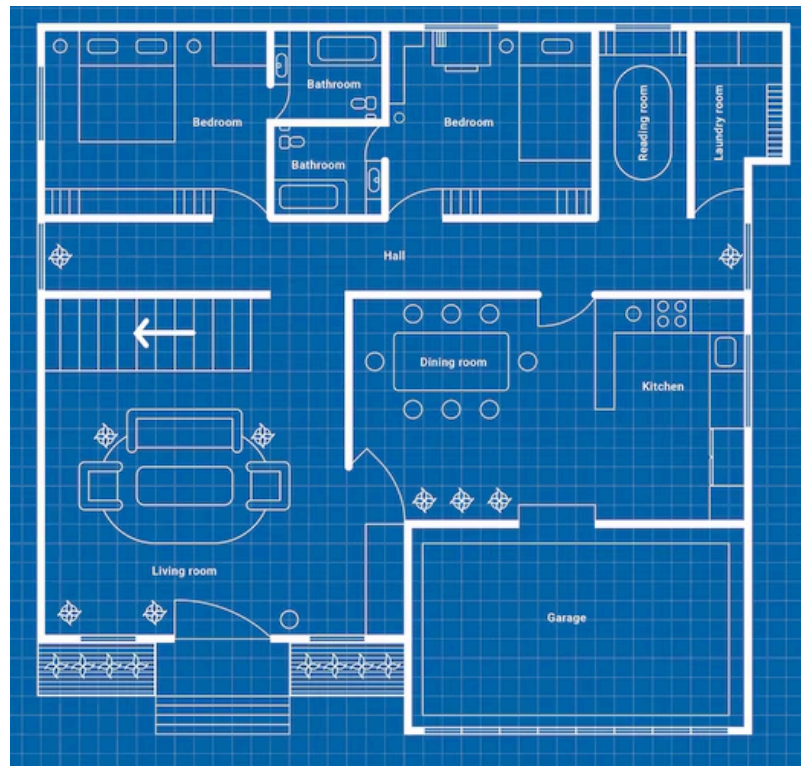


Figure 1: Blueprint of a house.

- The blueprint can be used to build any number of objects, or **instances**.



Figure 2: House objects build with a house blueprint.

- Example: What could you say about the Insect class and the two objects shown in the figure?

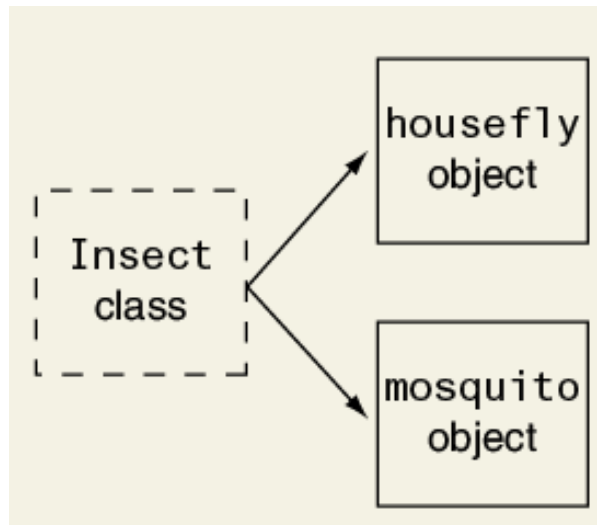


Figure 3: An Insect class and two of its instances.

Answer:

The Insect class could describe attributes and abilities of insects. The housefly object and the mosquito object are **instantiated** from Insect. Instances have all of the attributes and abilities described in the Insect class.

A lot of the code written for Insect applications can now be shared by applications for housefly and mosquito insects. When a housefly or mosquito are created, they get their own area in memory.

This makes conceptual sense: some properties/behavior is not shared between these - mosquitos and houseflies have different mouthparts, mosquitos sting and suck blood, houseflies feed by sponging.

9. Glossary - check your understanding of basic terminology and examples!

Term	Definition
Procedural Prog.	Programming with procedures and global data
OOP	Objects encapsulate data & logic
Encapsulation	Hide internal data; access via member functions
State/attribute	Data stored in an object

Term	Definition
Behavior	Actions defined by an object's methods
Car UI	Controls used (wheel, pedals, dashboard)
Car Implementation	Internal mechanisms (steering, fuel, brakes)
class (keyword)	Declares a user-defined type
class (code)	Contains attributes and member functions
class (concept)	Blueprint to create instances of an object
Object/Instance	Realized form of a class in memory
Insect (class)	Class describing common insect features
housefly/mosquito	Instances of Insect class

10. **PRACTICE** Create a Rectangle class

- Write a class Rectangle with:
 - private fields width, length
 - public methods:
 - setWidth
 - setLength
 - getWidth
 - getLength
- Test:
 1. Create a Rectangle object r
 2. Set width to 4
 3. Set length to 5
 4. Print width, length, and area of the rectangle
- Solution:

```
class Rectangle {  
    // private data  
    private double width;  
    private double length;  
    // public methods  
    public void setWidth(double w) {  
        width = w;  
    }  
  
    public void setLength(double l) {  
        length = l;  
    }  
  
    public double getWidth() {  
        return width;  
    }  
  
    public double getLength() {  
        return length;  
    }  
}
```

```

public class RectangleDemo {
    public static void main(String[] args) {
        // create a Rectangle object
        Rectangle r = new Rectangle();
        // set object properties
        r.setWidth(4.0);
        r.setLength(5.0);

        System.out.println("Width = " + r.getWidth());
        System.out.println("Length = " + r.getLength());
        System.out.println("Area = " + (r.getWidth() * r.getLength()));
    }
}

```

11. Separate interface and implementation in Java

- In C++, you used:
 - .h for declaration
 - .cpp for implementation
 - preprocessor to knit everything together
- In Java, the usual classroom equivalent is:
 - class signature and method declarations at the top
 - method bodies below
 - one public class per file
- Later, the stronger Java form is:
 - interface = API only, specifies **what**
 - class = implementation, specifies **how**
- Might give the stronger (production) file format as a bonus assignment (requires research).

12. Constructors

- A constructor initializes a new object.
- In Java:
 - constructor name = class name
 - no return type
 - called with new
- Notice that the Rectangle class did not have an explicit constructor method but only an implicit default constructor Rectangle().

13. Example: Counter constructors

- Final code in tinyurl.com/counter-demo-java

```

// Counter.java defines a counter
class Counter {
    private int count; // integer instance variable
    public Counter() {} // default constructor (count = 0)
    public Counter(int initial) {count = initial;} // alternate constructor
    public int getCount() {return count;} // accessor method
    public void increment() {count++;} // update method (add 1)
    public void increment(int delta) {count+=delta;} // update method
    public void reset() {count=0;} // update method
}
// CounterDemo.java tests the Counter class
public class CounterDemo {

```

```

public static void main(String[] args) {
    Counter c; // reference variable declaration (no counter)
    c = new Counter(); // construct new counter, assign ref to c
    c.increment(); // increase counter by 1
    c.increment(3); // increase counter by 3 (now it's 4)
    int temp = c.getCount(); // temp stores 4
    c.reset(); // counter is set back to 0

    System.out.println("Counter is " + temp);
}
}

```

14. PRACTICE Constructor for the Rectangle class

- Add and use **constructors** to properly initialize Rectangle objects.
- **Tasks:**
 1. Add a **default constructor** (no parameters) that:
 - sets width = 1
 - sets length = 1
 2. Add a **parametrized** constructor that:
 - takes two parameters: width and length
 - initializes the fields
 3. Add main to:
 - create one Rectangle using the default constructor
 - create one Rectangle using the parameterized constructor with width 4 and length 5
 - print the area of both rectangles
- **Hints:**

1. Constructor name must match the class name:

```
public Rectangle(...) { ... }
```

2. Constructors have no return type (not even void)

3. Use different parameter names than the fields to avoid confusion:

```

public Rectangle(double w, double l) {
    width = w;
    length = l;
}

```

4. The constructor is called automatically when you use new:

```
Rectangle r = new Rectangle(4.0, 5.0);
```

5. A default constructor has no parameters:

```

public Rectangle() {
    width = 1;
    length = 1;
}

```

- **Starter code:** tinyurl.com/rectangle-java

```
// Rectangle.java: set width and length and get area
class Rectangle {
    // private Rectangle data
    private double width;
    private double length;
    // public Rectangle constructors
    // ...
    // public Rectangle methods
    public void setWidth(double w) { // update width
        width = w;
    }
    public void setLength(double l) { // update length
        length = l;
    }
    public double getArea() { // get area
        return width * length;
    }
    // main method to test Rectangle class
    public static void main(String[] args) {
        // ...
    }
}
```

- **Sample output:**

```
Area of Rectangle: 1.0
Area of Rectangle: 20.0
```

- **Optional extension** (Bonus): Add a check that width and length must be > 0 . What should happen if invalid values are passed? (This will require some research on throwing exceptions.) Once you've added the guardrails, modify the program to accept keyboard input.

15. Solution: Constructor for the Rectangle class

Notice that there are two ways to test Rectangle: Either placing the main function inside the class definition, or separating the model Rectangle from the driver program RectangleDemo. This is what I do:

```
// Rectangle.java: set width and length and get area
class Rectangle {
    // private Rectangle data
    private double width;
    private double length;
    // public Rectangle constructors
    public Rectangle() {width=1;length=1;} // default
    public Rectangle(double w, double l){ // parametrized
        width=w; length=l;}
    // public Rectangle methods
    public void setWidth(double w) {width=w;} // update width
    public void setLength(double l) {length=l;} // update length
    public double getArea() {return width*length;} // get area
}
// RectangleDemo class to test Rectangle
public class RectangleDemo {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        Rectangle r2 = new Rectangle(4,5);
    }
}
```

```
        System.out.println("Area of Rectangle: " + r.getArea());
        System.out.println("Area of Rectangle: " + r2.getArea());
    }
}
```

16. Attributes: public / class / static / void

- Why is RectangleDemo a public class, but Rectangle is just a class (not public), and why is main a public static void function?
 - RectangleDemo is declared public because it is the main class of the program. In Java, the file name must match the public class name, and the JVM needs access to this class to start execution.
 - Rectangle is not public because it is only used as a helper class in the same file. Since it is not intended to be accessed from outside, it does not need to be public. Also, Java allows only one public class per file.
 - The main method is declared public static void because:
 - public: the JVM must be able to access it
 - static: it can be called without creating an object
 - void: it does not return a value

Summary:

- RectangleDemo = entry point of the program
- Rectangle = supporting class
- main = where execution begins

Author: Marcus Birkenkrahe

Created: 2026-04-25 Sat 08:17